COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 212	**SECTION:**	CPE31S2
DATE PERFORMED:	2026-10-20	**DATE SUBMITTED:**	2026-10-26
NAME:	San Juan, Ivan Wayne	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12 **Build a Sample Web App in a Docker Container**

1. PROCEDURE OUTPUT

Part 1: Launch the DEVASC VM



Figure 1.1 Launching the CyberOps Workstation VM

Part 2: Create a Simple Bash Script

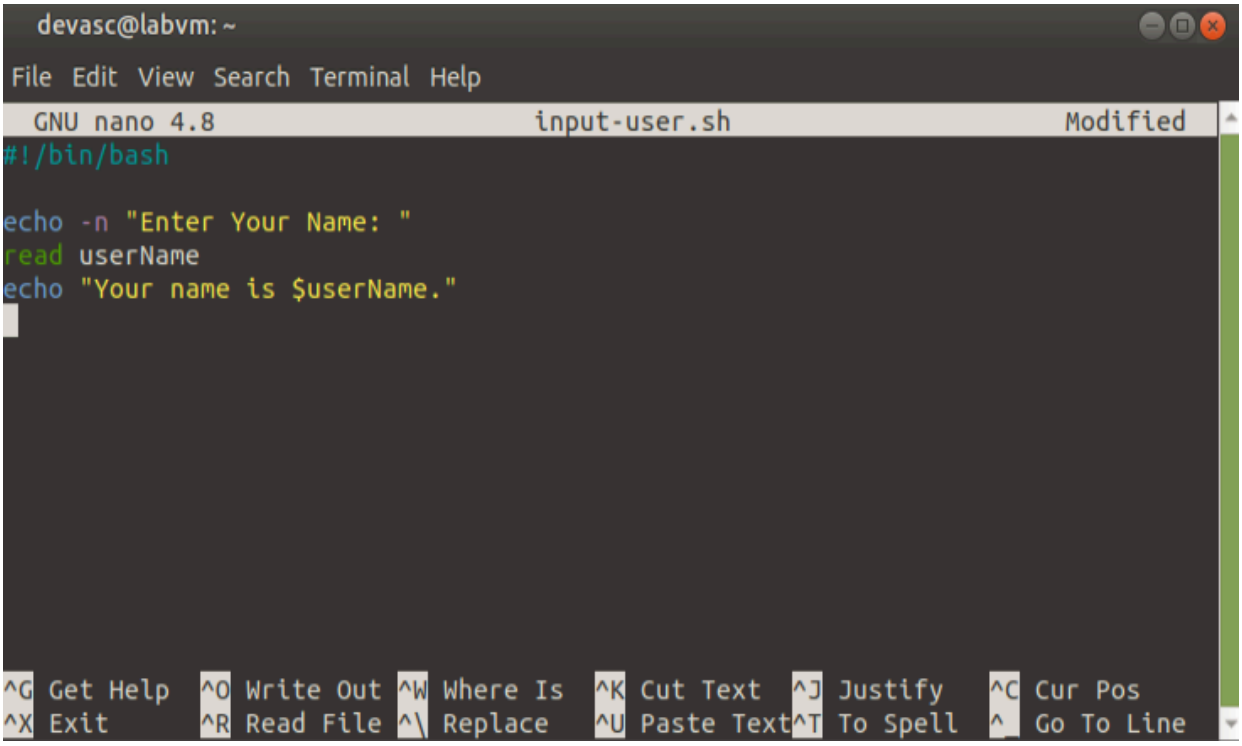


Figure 2.1. Bash Script

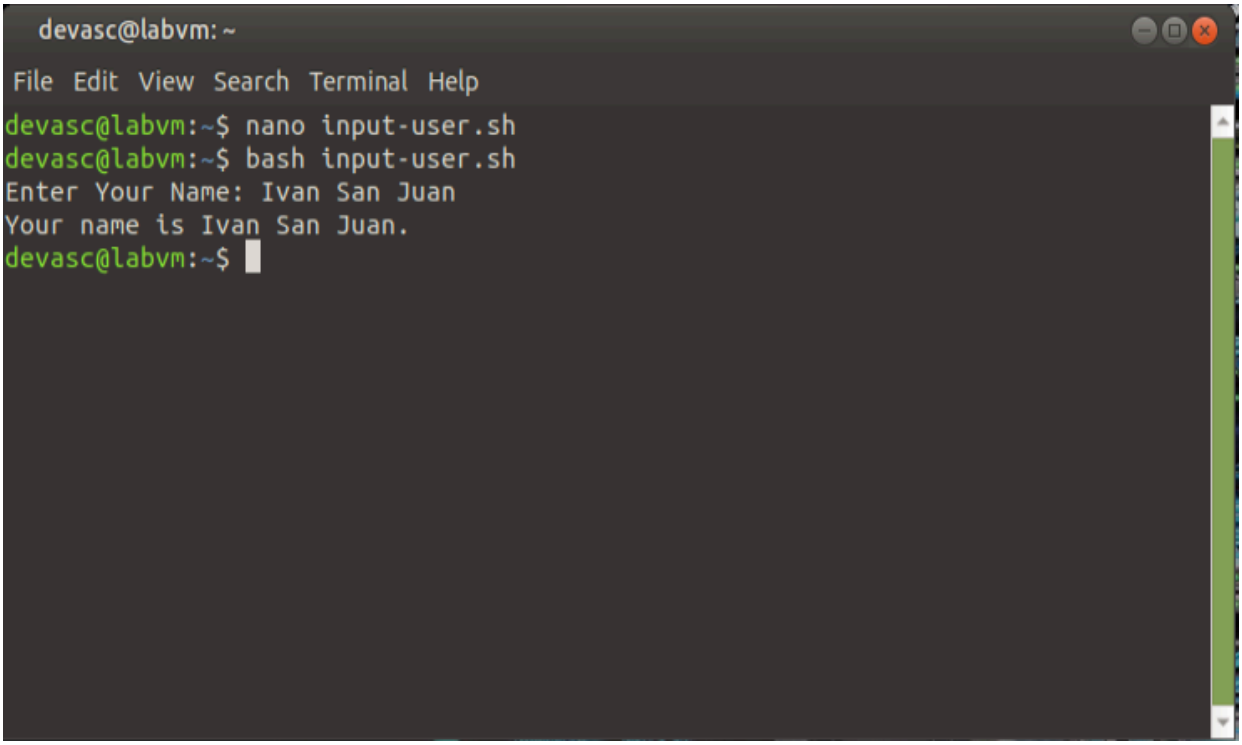


Figure 2.2. Bash user-input.sh

Part 3: Create a Sample Web App

- Step 1: Install Flask and open a port on the DEVASC VM firewall

```
devasc@labvm:~$ pip show flask
Name: Flask
Version: 1.1.2
Summary: A simple framework for building complex web applications.
Home-page: https://palletsprojects.com/p/flask/
Author: Armin Ronacher
Author-email: armin.ronacher@active-4.com
License: BSD-3-Clause
Location: /home/devasc/.local/lib/python3.8/site-packages
Requires: click, itsdangerous, Werkzeug, Jinja2
Required-by:
```

Figure 3.1. Installation of flask

- Step 2 - 6

```
devasc@labvm: ~
File  Edit  View  Search  Terminal  Help
  GNU nano 4.8                    sample_app.py
from flask import flask
from flask import request

sample = Flask(__name__)

@sample.route("/")
def main():
        return "You are calling me from" + request.remode_addr + "\n"

if __name__ == "__main__":
        sample.run(host="0.0.0.0", port=8080)




                        [ Read 12 lines ]
^G Get Help  ^O Write Out ^W Where Is  ^K Cut Text  ^J Justify   ^C Cur Pos
^X Exit      ^R Read File ^\ Replace   ^U Paste Text^T To Spell  ^_ Go To Line
```

Note: from flask import flask is wrong, it should be capital letter "f". So it's "from flask import Flask"

Figure 3.2. Python Code

- Step 7: Save and run your sample web app.

Figure 3.3. Running the sample_app.py

- Step 8: Verify the server is running.

```
devasc@labvm:~$ python3 sample_app.py
 * Serving Flask app "sample_app" (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
127.0.0.1 - - [28/Oct/2026 16:49:00] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [28/Oct/2026 16:49:06] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [28/Oct/2026 16:49:07] "GET / HTTP/1.1" 200 -
```

Figure 3.4. Calling 127.0.0.1

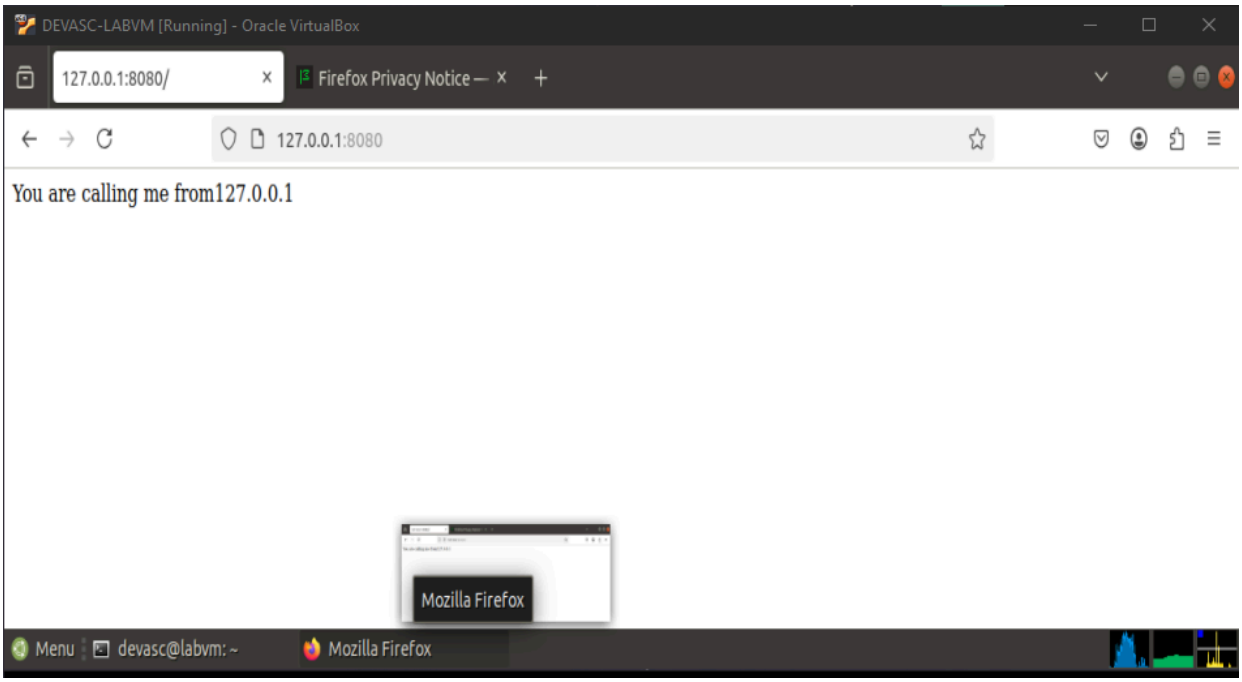


Figure 3.5. Verifying through web browser

```
devasc@labvm:~$ curl http://0.0.0.0:8080/
You are calling me from127.0.0.1
devasc@labvm:~$ curl http://0.0.0.0:8080/
You are calling me from127.0.0.1
devasc@labvm:~$ curl http://0.0.0.0:8080/
You are calling me from127.0.0.1
devasc@labvm:~$
```

Figure 3.6. Verifying through terminal

Part 4: Configure the Web App to Use Website Files

- Step 1: Explore the directories that will be used by the web app.

```
devasc@labvm:~$ cat templates/index.html
<html>
<head>
        <title>Sample app</title>
        <link rel = "stylesheet" href="/static/style.css" />
</head>
<body>
        <h1>You are calling me from {{request.remote_addr}}</h1>
</body>
</html>
```

Figure. 4.1. index.html

- Step 2: Update the Python code for the sample web app.

```
from flask import Flask
from flask import request
from flask import render_template

sample = Flask(__name__)

@sample.route("/")
def main():
        return "You are calling me from" + request.environ.get('REMOTE_ADDR', 'unknown') + "\n"

if __name__ == "__main__":
        sample.run(host="0.0.0.0", port=8080)
```

Figure 4.2. Updating the sample_app.py

- Step 3: Save and run your script

```
devasc@labvm:~$ python3 sample_app.py
 * Serving Flask app "sample_app" (lazy loading)
 * Environment: production
   WARNING: This is a development server. Do not use it in a production deployment.
   Use a production WSGI server instead.
 * Debug mode: off
 * Running on http://0.0.0.0:8080/ (Press CTRL+C to quit)
```

Figure 4.3. Running the sample_app.py

- Step 4: Verify your program is running.



Figure 4.4. Verifying through Browser

Part 5: Create a Bash Script to Build and Run a Docker Container

```
devasc@labvm:~/labs/devnet-src/sample-app$ cat sample-app.sh
cat > sample-app.sh << 'EOF'
#!/bin/bash

# Clean up previous runs to prevent errors if you run this twice
rm -rf tempdir

# Step 1: Create temporary directories
mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static

# Step 2: Copy the website directories and sample_app.py
cp sample_app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.

# Step 3: Create the Dockerfile
echo "FROM python" > tempdir/Dockerfile
echo "RUN pip install flask" >> tempdir/Dockerfile
echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY sample app.py /home/myapp/" >> tempdir/Dockerfile
```

```
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY sample_app.py /home/myapp/" >> tempdir/Dockerfile
echo "EXPOSE 8080" >> tempdir/Dockerfile
echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile

# Step 4: Build the Docker container
cat > sample-app.sh << 'EOF'
#!/bin/bash

# Clean up previous runs to prevent errors if you run this twice
rm -rf tempdir

# Step 1: Create temporary directories
mkdir tempdir
mkdir tempdir/templates
mkdir tempdir/static

# Step 2: Copy the website directories and sample_app.py
cp sample_app.py tempdir/.
cp -r templates/* tempdir/templates/.
cp -r static/* tempdir/static/.

# Step 3: Create the Dockerfile
```

```
# Step 3: Create the Dockerfile
echo "FROM python" > tempdir/Dockerfile
echo "RUN pip install flask" >> tempdir/Dockerfile
echo "COPY ./static /home/myapp/static/" >> tempdir/Dockerfile
echo "COPY ./templates /home/myapp/templates/" >> tempdir/Dockerfile
echo "COPY sample_app.py /home/myapp/" >> tempdir/Dockerfile
echo "EXPOSE 8080" >> tempdir/Dockerfile
echo "CMD python3 /home/myapp/sample_app.py" >> tempdir/Dockerfile

# Step 4: Build the Docker container
cd tempdir
docker build -t sampleapp .

# Step 5: Start the container and list it
docker run -t -d -p 8080:8080 --name samplerunning sampleapp
docker ps -a
EOF
```

Figure 5.1. Bash script for sample-app.sh

Part 6: Build, Run, and Verify the Docker Container

- Step 1: Execute the bash script.

```
devasc@labvm:~/labs/devnet-src/sample-app$ bash sample-app.sh
```

Figure 6.1. Executing bash script

- Step 2: Investigate the running Docker container and the web app.

```
devasc@labvm:~/labs/devnet-src/sample-app$ ls tempdir
Dockerfile  sample_app.py  static  templates
```

Figure 6.2. Verifying tempdir

```
devasc@labvm:~/labs/devnet-src/sample-app$ cat tempdir/Dockerfile
FROM python
RUN pip install flask
COPY ./static /home/myapp/static/
COPY ./templates /home/myapp/templates/
COPY sample_app.py /home/myapp/
EXPOSE 8080
CMD python3 /home/myapp/sample_app.py
```

Figure 6.3. Verifying tempdir/Dockerfile

```
devasc@labvm:~/labs/devnet-src/sample-app$ curl http://172.17.0.1:8080
<html>
<head>
    <title>Sample app</title>
    <link rel="stylesheet" href="/static/style.css" />
</head>
<body>
    <h1>You are calling me from 172.17.0.1</h1>
</body>
```

Figure 6.4. curl http://172.17.0.1:8080

```
</html>devasc@labvm:~/labs/devnet-src/sample-app$ ip address
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:e9:3d:e6 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
       valid_lft 82030sec preferred_lft 82030sec
    inet6 fd17:625c:f037:2:a00:27ff:fee9:3de6/64 scope global dynamic mngtmpaddr noprefixroute
       valid_lft 86391sec preferred_lft 14391sec
    inet6 fe80::a00:27ff:fee9:3de6/64 scope link
       valid_lft forever preferred_lft forever
3: dummy0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
    link/ether 52:b8:b9:28:31:ae brd ff:ff:ff:ff:ff:ff
    inet 192.0.2.1/32 scope global dummy0
       valid_lft forever preferred_lft forever
    inet 192.0.2.2/32 scope global dummy0
       valid_lft forever preferred_lft forever
    inet 192.0.2.3/32 scope global dummy0
```

Figure 6.5. ip address

- Step 3: Access and explore the running container.

```
root@c9b9d00c5b9a:/home/myapp# pwd
/home/myapp
root@c9b9d00c5b9a:/home/myapp# ls -l
total 12
-rw-r--r-- 1 root root  219 Oct 28 17:21 sample_app.py
drwxr-xr-x 2 root root 4096 Oct 28 17:30 static
drwxr-xr-x 2 root root 4096 Oct 28 17:30 templates
root@c9b9d00c5b9a:/home/myapp#
```

Figure 6.6. docker exec -it samplerunning bash

- Step 4: Stop and remove the Docker container.

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker ps -a
CONTAINER ID   IMAGE       COMMAND                  CREATED         STATUS                       PORTS     NAMES
c9b9d00c5b9a   sampleapp   "/bin/sh -c 'python3…"   4 minutes ago   Exited (137) 7 seconds ago             samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$
```

Figure 6.7. docker stop samplerunning

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker start samplerunning
samplerunning
```

Figure 6.8. docker start samplerunning

```
devasc@labvm:~/labs/devnet-src/sample-app$ docker stop samplerunning
samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$ docker rm samplerunning
samplerunning
devasc@labvm:~/labs/devnet-src/sample-app$ docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
devasc@labvm:~/labs/devnet-src/sample-app$
```

Figure 6.9. docker rm samplerunning

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Conclusion:

- This lab utilized a custom shell script to automate the deployment of a containerized Flask application, successfully navigating environment and Dockerfile constraints. Ultimately, we achieved a reliable, automated setup that continuously serves HTTP requests on port 8080.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.